Magertron™

AI Orchestrator
Getting Started Guide

From installer to your first governed MCP server

Quick-Start Edition

Contents

Every line above is a live link — click a chapter or section to jump straight to it.

About this guide

This is a quick-start walkthrough covering the happy path from installation to your first registered server. It is intentionally not a complete reference — for the full operational picture, see the *Magertron AI Orchestrator Operator's Guide*.

Document type	Operator's guide
Audience	First-time operators standing up Magertron in a Kubernetes cluster
Status	Quick-start edition — the fastest path from an empty cluster to a registered, governed MCP server. This guide deliberately covers the happy path only; for the complete operational picture, see the Operator's Guide.

How to read this guide. This is a hand-held walkthrough, not a reference. Follow the chapters in order and you will go from a fresh cluster to a live MCP server in well under an hour — installing with the prescribed installer, logging in, securing the bootstrap admin account, and registering your first server. Each step ends in a verifiable checkpoint, so you always know whether to proceed. Where a topic has more depth than a quick start should carry, you will see a pointer to the relevant chapter of the Operator's Guide rather than a detour.

Chapter 1 — Before you begin

This guide takes you from nothing to a working, governed MCP server. The whole path is four moves: install, log in, secure the admin account, and register a server. Everything else in the platform builds on this foundation, so getting these four right is the entire goal of the quick start.

1.1 What you will accomplish

By the end of this guide you will have a running Magertron control plane in your own cluster, an admin session you have secured, and one MCP server registered, approved, and serving governed traffic. That is the complete first-win loop. The cost, metering, and governance capabilities that make Magertron worth running all sit on top of this loop — once a server is registered and traffic flows through the gateway, those capabilities have something to measure.

1.2 What you need on hand

A short checklist before you start. None of this is exotic; most evaluation clusters already have it.

- A Kubernetes cluster (1.28 or newer) you can reach. Confirm with `kubectl get nodes`. The reference deployment runs on k3s, which is the recommended choice for a single-box evaluation.

- Helm 3.x and kubectl on your workstation. Confirm with `helm version` and `kubectl version --client`.
- A PostgreSQL-capable storage class in the cluster. The orchestrator and inventory databases each need durable storage; the chart defaults to a 10 Gi volume for each.
- A way to reach the gateway. For a single-box install, the NodePort path needs nothing extra — you forward external 443 to the cluster node's NodePort. Production ingress choices are covered in the Operator's Guide (Chapter 15).

For an evaluation, the chart can auto-generate TLS certificates, JWT signing keys, and the database and admin passwords. That is exactly what you want for a first run — it removes every prerequisite except the cluster itself. Production hardening (bring-your-own certificates, keys from a KMS, and so on) is a separate exercise covered in the Operator's Guide; do not let it slow down your first install.

> **A note on k3s.** Two k3s defaults trip up first installs; set both at install time and neither will bite you. First, disable bundled Traefik — otherwise Traefik and Envoy both try to bind port 443 and one of them will fail. Second, make k3s write a readable kubeconfig — by default it writes `/etc/rancher/k3s/k3s.yaml` root-owned and mode `600`, so a non-root `kubectl get nodes` (and `install.sh`'s cluster preflight) fails with a "permission denied / cannot reach cluster" error. Install with both flags at once:
>
> ```
> curl -sfL https://get.k3s.io | sh -s - --disable=traefik --write-kubeconfig-mode 644
> ```
>
> If k3s is already installed without the kubeconfig flag, you have two ways to fix it without reinstalling. The fastest is to make the existing file readable in place:
>
> ```
> sudo chmod 644 /etc/rancher/k3s/k3s.yaml
> kubectl config current-context
> kubectl get nodes
> ```
>
> That clears it immediately. Note that `644` lets any user on the box read the cluster-admin credential — acceptable on a single-operator evaluation or demo box you control, but not on a shared or production host. For a cleaner posture, copy the config into your home directory and take ownership instead, leaving the system file locked down:
>
> ```
> mkdir -p ~/.kube
> sudo cp /etc/rancher/k3s/k3s.yaml ~/.kube/config
> sudo chown $(id -u):$(id -g) ~/.kube/config
> unset KUBECONFIG   # in case it points at the root-owned file
> ```
>
> then confirm with `kubectl config current-context` (it should read `default`) followed by `kubectl get nodes`. Between Traefik and the kubeconfig permissions, these are the two most common first-install snags on k3s.

1.3 Confirm you can reach the cluster

Three of the prerequisites in §1.2 are worth confirming from the command line before you go any further. The installer runs its own preflight and will tell you the same things, but it tells you later, and a cluster you cannot reach looks very like a platform that will not install.

> **A note on kubectl.** k3s installs its own client at `/usr/local/bin/kubectl`, already configured for the cluster it just created — use it. If you install `kubectl` separately, match your cluster's minor version. A client more than one minor version away from the API server is unsupported, and it fails in ways that look like cluster problems rather than version skew:
>
> ```
> kubectl version
> kubectl get nodes
> ```
>
> If `kubectl` cannot reach the cluster at all, suspect the kubeconfig rather than the cluster — and prefer the copy-and-own recipe in §1.2 to loosening permissions on a host you share with anyone.

The other two are one command each:

```
helm version --short      # Helm 3.x on your workstation
kubectl get storageclass  # at least one, and one marked (default)
```

> **Checkpoint.** `kubectl get nodes` returns your node as `Ready`, Helm reports 3.x, and a default storage class exists. That is the whole prerequisite list — a PostgreSQL pod stuck `Pending` in Chapter 2 is almost always the storage class that was never confirmed here.

1.4 Hardware requirements

Magertron is an enterprise-class, multi-component platform — a C++ control plane (2 replicas), an Envoy data plane (2 replicas, autoscaling to 10), two PostgreSQL databases, and the inventory subsystem, all on top of a Kubernetes distribution. Your hardware budget is the sum of those workloads. The figures below are hard floors for the whole cluster, not aspirational targets; provision at or above the recommended row for anything a real workload or a buyer will touch.

Every node that will schedule Magertron workloads must meet at least the minimum row. A single-node (k3s) install must meet the recommended single-node row, because that one node carries every component at once.

Profile	vCPU (cores)	RAM	Free disk (SSD)	Notes
Absolute minimum per node	4 cores	16 GB	50 GB	Floor for a node to schedule Magertron components at all. Below this, pods will fail to schedule or be OOM-killed.
Recommended single-node (k3s, evaluation)	8 cores	32 GB	100 GB	One box runs orchestrator ×2, Envoy ×2, PostgreSQL ×2, inventory, and the K8s control plane simultaneously. The realistic floor for a working demo or design-partner install.
Recommended per worker node (multi-node production)	8+ cores	32+ GB	200+ GB SSD	Production worker. Add nodes to scale horizontally rather than oversizing one.
Control-plane node (multi-node)	4 cores	16 GB	100 GB	Dedicated kubeadm control-plane node.

Architecture requirements

- **CPU architecture:** x86-64 (amd64). ARM64 is not yet exercised end to end; 32-bit and other architectures are unsupported.
- **Operating system:** a 64-bit Linux distribution capable of running a conformant Kubernetes (k3s, kubeadm, RKE2). Ubuntu 22.04+ / Debian 12+ are the exercised baselines.
- **Storage:** SSD-backed, not spinning disk or eMMC. PostgreSQL write latency on the metering and audit streams is the first thing to degrade on slow storage. The disk figures above include headroom for both databases, container images, and logs.
- **Networking:** a stable private network between nodes. A 1 Gbps link is sufficient for most deployments; a 10 GbE backbone is recommended for high-request-rate production where gateway-to-orchestrator and orchestrator-to-database traffic is heavy.

> **Will not run on.** Magertron is not a laptop or edge-device application. It will not run on a Chromebook, a Raspberry Pi or similar SBC, a sub-4-core or sub-16 GB VM, or any single-board / thin-client device — these lack the CPU, memory, and disk to bring up a full Kubernetes cluster plus the five-component platform, and an install attempt will fail at pod scheduling or first boot. If you are evaluating on a single machine, use the recommended single-node profile above on a proper Linux host or VM. Full per-component sizing guidance is in the Operator's Guide (Chapter 16).

1.5 Measured resource footprint

The table in §1.4 gives hard floors for the cluster. This section shows the platform's *actual* measured RAM and container-image footprint, so you can right-size for the specific configuration you intend to run. Two subsystems — the local LLM (Ollama) and embedded GRC (Probo) — are optional and dominate their respective footprints; whether you enable them changes the numbers dramatically.

Figures are measured on the reference single-node lab deployment, at rest. The core platform is the orchestrator HA pair, the Envoy data plane (legacy + v3), the inventory service, the primary and inventory PostgreSQL databases, and the mail relay.

Configuration	Resting RAM	Image disk	Notes
Core only (no local LLM, no embedded GRC)	~259 Mi	~505 MB	Full orchestration + governance control plane. Use the Anthropic-backed assistant, or run without the assistant.
Core + embedded GRC (Probo)	~459 Mi	~609 MB	Adds SOC 2 / compliance tooling.
Core + local LLM (Ollama)	~274 Mi resting; **+4–5 GB when a model is loaded**	~3,775 MB	The LLM model dominates both RAM (under load) and disk.
All subsystems	~474 Mi resting; **+4–5 GB under local-LLM load**	~3,775 MB	Full platform as measured (16 pods).

Size for the peak, not the idle floor. The RAM figures above are *resting*. The single biggest sizing driver is the local LLM: idle Ollama uses only ~15 Mi, but running a local inference loads the model into RAM — a 7B-class model (e.g. Qwen 2.5 7B) needs roughly **4–5 GB of additional memory while resident, even quantized**. If you enable the local assistant, size the node for the loaded model, not the idle pod. If you use the Anthropic-backed assistant instead, Ollama can be omitted entirely — you avoid both the ~3.27 GB image pull (about 87% of the total disk footprint) and the multi-GB RAM cost under load.

> **Enabling the optional subsystems.** Both are off by default. To deploy them, pass `--enable-ollama` (the local LLM assistant) and/or `--enable-probo` (embedded GRC) to `install.sh` at install time — for example `./install.sh --enable-ollama --enable-probo`. Size the node for the footprint above before enabling either. Full sizing, prerequisites, and the Anthropic-vs-Ollama assistant choice are covered in the Operator's Guide.
>
> **Note.** These figures cover RAM and container-image disk only. Provision CPU and data-volume disk for your own throughput and retention separately, and expect PostgreSQL to grow with working-set size under real traffic. Development clusters also accumulate superseded image tags that inflate on-disk usage beyond the numbers above; prune them with your container runtime's image-removal command. Measure your own deployment with `kubectl top pods -n mcp-system` (memory) and your runtime's image list (disk) after a representative load.

Chapter 2 — Install with `install.sh`

Magertron ships a bundled installer, `install.sh`, and it is the prescribed install path — not raw `helm install`. The same script handles both first-time install and upgrade-in-place. It exists because it does several things correctly that a hand-written Helm command leaves you to get right, two of which fail silently if you miss them. For a quick start, the only thing you need to know is: run the script, answer one prompt, and let it work.

2.1 Get the installer

Everything you need ships in one repository — the installer, the Helm chart, and the supporting files. Clone it and step into the directory:

```
git clone https://github.com/magertron/orchestrator.git
cd orchestrator
```

Make the installer executable, then register the Magertron Helm chart repository so the installer can resolve the chart:

```
chmod +x ./install.sh
helm repo add magertron https://magertron.com/charts
helm repo update
```

That is the same one-liner shown on the Magertron home page. From here, a complete free-tier install is a single command — and if you have an Enterprise license, you point the installer at it:

```
./install.sh                                  # Free tier — no license needed
./install.sh --license /path/to/license.json  # With a license
```

The next section walks through that run.

2.2 Run the installer

From the chart directory, the free-tier evaluation install is a single command with no flags:

```
# Free-tier evaluation — interactive, no license required
./install.sh
```

The script runs interactively and prompts you for one value it cannot infer on its own: the external public URL — the externally-reachable HTTPS address a browser would use to reach the orchestrator (for example, `https://mcp.yourcompany.com`, or the node address you will forward 443 to). Enter it carefully. This single value drives the OAuth callback target later; if it is wrong, browser-based logins redirect to an address they cannot reach, and the failure does not surface until someone tries to log in. The installer validates the format for you, which is most of why the prescribed path exists.

```
[10/15] Helm install
╔═══════════════════════════════════════════════════════╗
║  IMPORTANT: Public URL for OAuth & DCR callbacks      ║
╚═══════════════════════════════════════════════════════╝

Magertron needs the externally-reachable HTTPS URL that an
end-user's browser uses to reach this orchestrator. It builds
the delegated-OAuth / DCR callback target from it:

    <your-url>/api/v1/oauth/callback

Why this matters: OAuth providers redirect the user's browser
back to this exact URL after they sign in. If it's wrong or
unset, the browser is sent to a localhost/internal address it
cannot reach — and every delegated-OAuth and DCR login fails
with a dead redirect. This is the one value the installer
cannot infer for you.

Examples:
  Production:  https://magertron.example.com
  Dev/tunnel:  https://your-name.ngrok-free.dev

(No scheme guess, no trailing slash — paste the full URL.)

Public HTTPS URL: _
```

The `install.sh` public-URL prompt. This is the one value the installer cannot infer — enter the full `https://` URL with no trailing slash.

If you already have an Enterprise license file, point the installer at it instead; everything else about the run is identical:

```
# First-time install with a license
./install.sh --license ~/Downloads/license.json
```

2.3 Watch the rollout

You do not have to babysit kubectl here — the installer waits for the control plane to come up and reports each step as it goes. It brings up the orchestrator, the Envoy gateway (both the legacy and v3 data planes), and two PostgreSQL databases (one for the orchestrator, one for the inventory service), pins the gateway to NodePort 30444, and prints the URL you will use to reach the platform.

```
kubectl get pods -n mcp-system -w
```

The command above is there if you want to watch pods come up in a second terminal, but the installer's own output is the authoritative signal — it tells you when the rollout is complete and every pod is ready:

```
[11/15] Waiting for orchestrator rollout
                                                  
✓ Rollout complete · 10/10 pods ready (41s)

[12/15] Pinning Envoy NodePort to 30444
                                                  
✓ NodePort pinned to 30444

[14/15] Final cluster state
                                                  
NAME                                          READY   STATUS    AGE
mcp-orchestrator-5869db5c8-822jl              2/2     Running   45s
mcp-orchestrator-5869db5c8-tgmrz              2/2     Running   45s
mcp-orchestrator-envoy-6f7bf8b6f9-kp8k6       1/1     Running   45s
mcp-orchestrator-envoy-6f7bf8b6f9-qvmx7       1/1     Running   45s
mcp-orchestrator-envoy-v3-55777f6fcb-tszgw    1/1     Running   45s
mcp-orchestrator-envoy-v3-55777f6fcb-w4p4h    1/1     Running   45s
mcp-orchestrator-envoy-v3-55777f6fcb-wkmnj    1/1     Running   45s
mcp-orchestrator-inventory-69794dfcc7-qmtr7   1/1     Running   45s
mcp-orchestrator-inventory-postgres-...-bkhjt 1/1     Running   45s
mcp-orchestrator-postgres-5676546b68-7c8h4    1/1     Running   45s

[15/15] Access
                                                  
UI / API:  https://192.168.1.124:30444
(TLS is self-signed; use -k with curl or accept the cert warning.)

╔═══════════════════════════════════════════════════╗
║  ✓ Install complete                               ║
║                                                   ║
║   URL            https://192.168.1.124:30444      ║
║   Data preserved ✓ all PVCs intact                ║
║   JWT keypair    ✓ preserved across helm upgrade  ║
╚═══════════════════════════════════════════════════╝
```

The tail of the install — rollout, final cluster state, and the success summary. Ten pods reach Ready, the gateway is pinned to NodePort 30444, and the UI/API URL is printed.

> **Checkpoint.** When the installer reports *Rollout complete* and prints the Access summary with your UI/API URL (for example `https://192.168.1.124:30444`), the platform is up. The TLS certificate is self-signed for an evaluation install, so your browser will warn on first visit — that is expected; accept it to proceed. Keep the installer's output on screen: the post-install summary points you to the bootstrap admin password you will need in the next chapter. If a PostgreSQL pod is stuck *Pending*, it is almost always the storage class — confirm one is available and is the default.

2.4 One port, every surface

The installer pins a single NodePort — 30444 unless you say otherwise — and every surface of the platform sits behind it. The admin UI, the developer portal, the REST API and the MCP gateway are paths on that one port, not separate listeners, so once the sign-in page answers, all of them do:

```
https://<node-ip>:30444/                        # admin UI
https://<node-ip>:30444/portal                  # developer portal
https://<node-ip>:30444/api/v1/...              # REST API
https://<node-ip>:30444/mcp/<ns>/<server>/      # MCP gateway
```

If you pinned a different port with `install.sh --node-port <n>`, every path above moves with it. Before you go hunting for a networking problem, confirm what is actually listening:

```
kubectl get svc -n mcp-system | grep NodePort
```

2.5 Two things worth setting during install

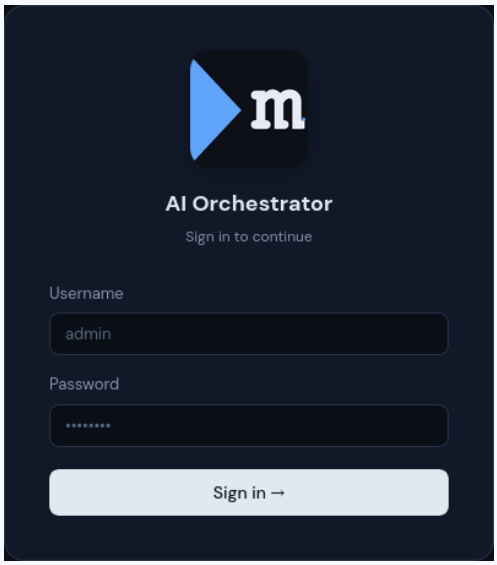
Neither is required to finish this guide, and both cost far more to change later than to set now.

The public URL. The installer prompts for it in §2.2, and the answer is baked into the delegated-OAuth callback. On an evaluation box the node's own address is fine (`https://192.168.1.x:30444`); on anything longer-lived, give it the hostname you actually intend to keep. Changing it afterwards means revisiting the callback in more than one place.

An admin email. The seeded administrator has none, and on a Free install with no identity provider that leaves no password-recovery path at all. It matters again later: if you add SSO, an admin with no email will not match the identity your IdP returns, and you end up with two accounts where you meant to have one. Set it at first login, alongside the password change in §3.3.

Chapter 3 — Log in and secure the admin account

The platform is running, but the only way in is a one-time bootstrap admin password that Kubernetes generated during install. Your job in this chapter is to log in with it once and immediately replace it — treat the bootstrap value as spent the moment you have used it.



Sign in at your orchestrator URL — use the URL printed by the installer, e.g. `https://192.168.1.124:30444`

3.1 Retrieve the bootstrap admin password

On first install, Magertron seeds a single administrator account and stores its password in a Kubernetes Secret. Retrieve it with the command the installer printed (it reads the Secret and decodes it):

```
kubectl get secret -n mcp-system mcp-orchestrator-secrets \
  -o jsonpath='{.data.MCP_SEED_ADMIN_PASSWORD}' | base64 -d
```

That prints the bootstrap password to your terminal. This is a deliberately one-time credential: the seed logic only runs against an empty database, so this value exists to get you in the door exactly once.

3.2 Log in

Open the orchestrator URL from the installer's summary in your browser and sign in as `admin` with the bootstrap password. The admin UI is where you will register servers, read dashboards, and manage roles.

Operators who prefer the terminal can log in with the CLI instead. If you have not installed it yet, `mcpctl` is free for all tiers — the Homebrew or curl one-liner is the quickest:

```
# Install the CLI (macOS or Linux)
curl -fsSL https://magertron.com/install-mcpctl.sh | sh

# Point it at your orchestrator and log in
mcpctl login https://<orchestrator-host>:30444 admin
```

The CLI caches your session at `~/.mcpctl/config.yaml`; delete that file any time you want to start fresh.

The Magertron admin UI sign-in screen. Enter `admin` and the bootstrap password; the "Sign in with auth0-test" button appears when an OIDC identity provider is configured.

3.3 Change the password immediately

This is the one step in the quick start you must not skip. From the admin UI, change the admin password now, on first login. The bootstrap value has served its purpose; rotate it to something only you hold.

Because the seed code only runs against an empty database, changing the password in the UI is sufficient — the bootstrap Secret value becomes inert and changing it afterward does not affect the running admin account. Set a strong password, store it in your password manager, and move on.

> **Checkpoint.** You are logged in as admin with a password you chose, and the bootstrap credential is retired. You now have a secured control plane and an authenticated session. The next section puts that session to work.

3.4 Giving the assistant a model

The assistant is docked in the admin UI and knows a great deal about your platform — what is registered, what it cost, what changed in the last day. What it does not have on a fresh install is a model to think with.

Magertron will not guess which one. Choosing a model provider means choosing where your prompts go and whose bill they land on, so the platform asks rather than assumes.

Three steps, and then it works.

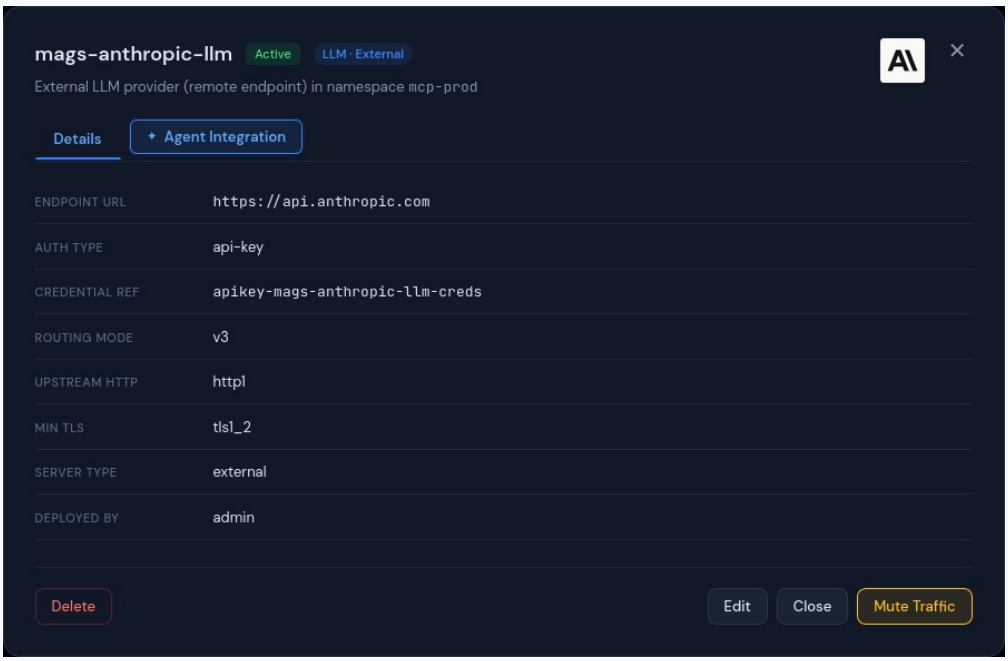
1. Register the model provider

Servers → Add Server → LLM.

You supply the endpoint and the vendor key. The key is stored by the platform, encrypted, and injected into each call on the way through — the assistant never holds it, and neither does anything else you connect later.

The examples in this guide use a provider named `mags-anthropic-llm`.

The registered provider, `mags-anthropic-llm` — an external LLM in `mcp-prod`. The endpoint and auth type are recorded on the server; the vendor key itself is held as a credential reference rather than shown.



Once it is registered it behaves like any other server on the platform. Its calls are metered, they land on a contract, and they appear in the audit trail with the principal that made them.

2. Grant the assistant access to it

Tool Explorer → your LLM server → grant `system:llm-consumer`.

The assistant runs as `mcp-assistant-sa`, which holds `system:llm-consumer`. That role makes a principal *eligible* to call a governed model provider — it grants no particular one.

So registering a provider does not, by itself, let anything call it.

> **This is deliberate, not an oversight.** `system:llm-consumer` is a role your own service accounts can hold too. A registration that automatically granted every holder access to a new provider would mean nobody decided who may spend against that vendor. Which agents reach which models is a decision, and the platform asks you to make it.

The same applies to any agent you build later: give it the role, then grant it the providers it should reach.

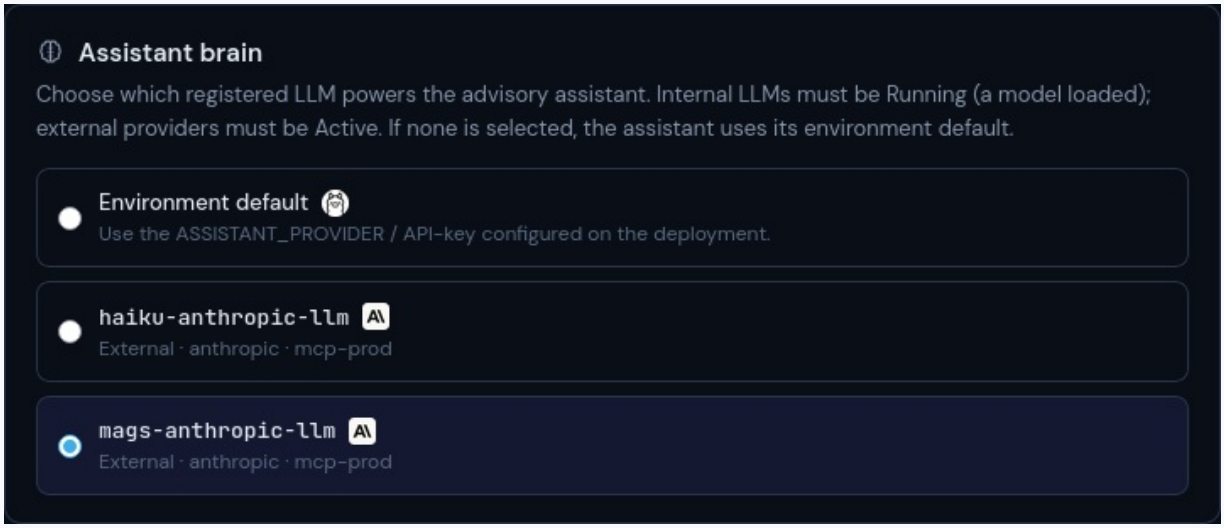
Until you grant it, the assistant reports that Magertron refused its model call and names this grant as the reason.

3. Select it as the assistant's brain

Settings → Assistant.

Your newly registered provider appears in the list. Select it.

Settings → Assistant, with `mags-anthropic-llm` selected. Internal providers must be Running and external ones Active to be eligible; with nothing selected the assistant falls back to the deployment's environment default.



The platform now knows which model the assistant should use, and the assistant authenticates as itself when calling it — so its own model calls are authorized, metered and recorded exactly like any other principal's work. The assistant is not exempt from the platform it runs on.

4. Try it

Open the assistant and ask something that needs a lookup rather than a definition:

> *What was the most recent denied call on the platform?*

A good answer reaches for the evidence server, finds the exchange, fetches the authority record, and tells you not just what was refused but which rules were in force when it happened.

If instead it explains where in the UI you could find that yourself, one of the three steps above is incomplete — most often the grant.

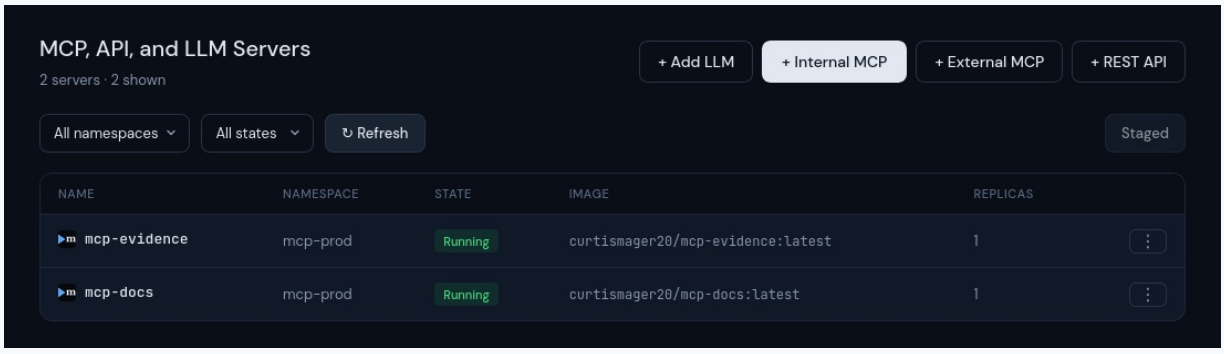
Do not delete mcp-docs or mcp-evidence

Both servers arrive with the platform and both are load-bearing.

`mcp-docs` is how the assistant answers questions about how Magertron works. `mcp-evidence` is how it answers questions about what happened — which exchange, which principal, what the rules were at the time.

Without them the assistant can still describe the platform's current state, because that is computed for it on every request. It cannot look anything up. It cannot tell you why a call failed on Tuesday, or what a service account was permitted to reach last week. It will not say so plainly either — it will simply answer worse, and point you at screens instead.

Both servers `Running` in `mcp-prod` on a fresh install. They appear in the server list, and behave in it, exactly like any other internal server.



They are ordinary servers in every other respect: they appear in the server list, their calls are metered, their tools are governed, and the audit trail records what the assistant does with them exactly as it records any other agent's work. Nothing about being the platform's own servers exempts them from the gate.

If one is deleted, restarting the orchestrator provisions it again — provisioning runs at every start and acts only on what is missing. But the grants and approvals that went with it do not come back on their own, so the tidier course is not to delete them.

Chapter 4 — Register your first MCP server

This is the payoff. A server is how tools enter the platform, and registering one is what turns Magertron from "installed" into "governing traffic." There are two kinds of server, and the distinction shapes everything downstream.

4.1 Two kinds of server

An **internal server** is an MCP server that Magertron deploys and runs as a pod inside your cluster — you give it a container image and the platform runs it. An **external server** is a third-party, vendor-hosted MCP endpoint that Magertron fronts rather than runs — you give it an endpoint URL and a credential reference, and the platform sits in front of it.

For your first registration, an internal server is the simplest possible win: no vendor credentials to arrange, no external endpoint to reach. We will use that path here.

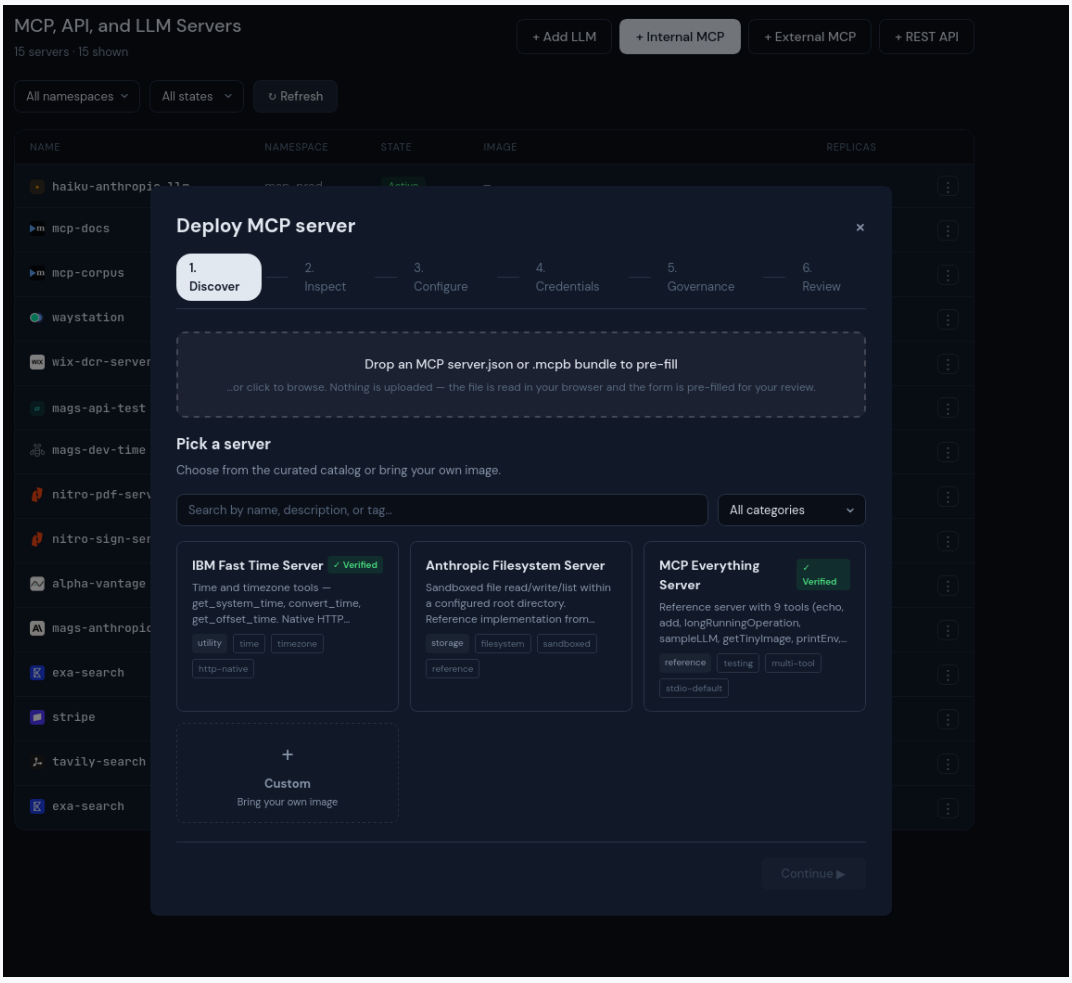
4.2 Register an internal server

From the admin UI, the Deploy Wizard walks you through registration in five steps — **Discover, Inspect, Configure, Governance, Review** — validating as you go and running a governance pre-check before anything deploys. It is the recommended path for your first server. The equivalent from the CLI is a single command:

```
# Deploy an internal MCP server
mcpctl deploy <name> <namespace> <image>
```

The default namespaces created at install are `mcp-prod`, `mcp-staging`, and `mcp-dev`; `mcp-dev` is the natural home for a first experiment. The deploy command accepts flags for the finer points — `--port`, `--transport`, `--upstream-path`, `--replicas`, and `--wait` among them — but the three positional arguments are all you need to get started.

The Deploy Wizard's final Review step. The five-step flow — Discover, Inspect, Configure, Governance, Review — ends here, summarizing the source, image, and configuration, with a governance pre-check showing All policies pass before you click Deploy.



Deploy Internal and External MCP and LLM Servers as well as external API endpoints.

4.3 Watch it come up

Because this is an internal server — one Magertron deploys and runs itself — there is no separate approval step to perform. The platform pulls the image, starts the pod, programs the gateway with its route, and the server comes up **Running** on its own. (Approval is part of the lifecycle for external, vendor-hosted servers, where an operator vets the endpoint and its credentials before traffic is allowed; see the note below.)

Open the server in the admin UI and you will see it land in the **Running** state, with its full spec and live health at a glance — transport, port, replicas, resource limits, and passing liveness/readiness checks.

The server detail view for a freshly deployed internal server, `mags-dev-time`, in the Running state. The Overview tab shows its spec (transport, port, replicas, limits) and live health — liveness and readiness both passing, with current CPU and memory. The Scale replicas control and Edit / Restart / Undeploy actions sit at the bottom.

4.4 Confirm it from the CLI

You can confirm the same thing from the terminal — list your servers and check that the one you just deployed is up:

```
# List deployed MCP servers
mcpctl servers
```

The admin UI shows the same fleet view. Notice the **State** column: your internal server (`mags-dev-time`) reads Running, as does any other server Magertron hosts as a pod, while external, vendor-hosted servers (`stripe`, `context7`, `tavily-search`, and so on) read Active — the two states that distinguish the two kinds of server you met in §4.1.

The MCP servers list in the admin UI. Internal servers Magertron hosts as pods — documentation and your new `mags-dev-time` — show the Running state with their container image; external, vendor-hosted servers show Active. Filters for namespace and state, plus deploy actions, sit along the top.

> **Checkpoint.** Your server is Running and its health checks pass. Agent traffic addressed to it now flows through the Magertron gateway, where the platform authorizes every call and routes it to the tool. You have completed the first-win loop: installed, secured, and governing live traffic.
>
> **A note on external servers.** The server you just deployed is internal, so it went straight to Running. An external (vendor-hosted) server follows a slightly longer path — register, then **approve** it, after which it converges to Active once the gateway has its route and credential material on every replica. That flow, and the credential handling behind it, is covered in the Operator's Guide (Chapters 5 and 6).

Chapter 5 — What you just unlocked (and where to go next)

You now have a running, secured Magertron control plane with a server registered and serving governed traffic. That is the foundation — and it is also the point where the platform's real value begins, because everything that

makes Magertron a spend control plane sits on top of the loop you just completed.

5.1 The capabilities now within reach

With traffic flowing through the gateway, these are the next things to explore — each is a chapter of its own in the Operator's Guide:

- **Metering** — every allowed call your server handles is now meterable, recorded once and attributable to the principal who made it. This is the raw material for everything financial.
- **Rating** — turn metered usage into money to the penny, with per-tool rate cards, tiered pricing, and time-selected schedules. A new customer deal becomes a row of data, never a recompile.
- **Cost attribution and chargeback** — attribute rated spend to the department and user that incurred it, point-in-time, exactly the way a finance organization expects.
- **Spend throttling** — set a per-server spend ceiling and have the platform block calls once it is reached, with an admin override path. The CFO's guardrail, not a kill switch.
- **Access control and credentials** — per-tool RBAC so a role grants specific tools rather than blanket access, and BYOK credential handling across five auth protocols for the external servers you register next.

None of these required anything beyond the loop you just finished. They were waiting for traffic to govern, and now there is some.

5.2 This was the quick start — the manual goes deeper

This guide did one thing on purpose: get you to a working, governed server as fast as honestly possible, and no further. It is the happy path, the first thirty minutes, the proof that the platform installs and runs and does what it says. It is not the whole story, and it was never meant to be.

For everything past this point — the cost engine in depth, external-server registration with real vendor credentials, delegated per-user identity, SSO and SCIM, audit and webhooks, day-2 operations, hardware sizing, and the full configuration surface — refer to the *Magertron AI Orchestrator Operator's Guide*. It is the complete operational reference, and it picks up exactly where this quick start leaves off.

Welcome to Magertron. You are up and running.

Chapter 6 — Where the operator stops and the developer starts

Everything up to this point has been the operator's job: install the platform, sign in, register a server, watch it come up. That work is done once.

What follows is for a **different person** — the developer who wants to browse what the platform governs and actually call a tool. They do not install anything, they do not need the admin UI, and most of this guide does not apply to them. Two things stand between them and a working session, and both are yours to do.

6.1 Create the developer's account

A developer cannot self-serve. The account and its roles are created by an administrator, in **Settings → Users**. Three roles matter, and they are not interchangeable:

Role	What it grants
`system:mcp-consumer`	Invoke MCP tools through the gateway
`system:llm-consumer`	Reach governed LLM providers
`system:api-consumer`	Reach governed, metered REST APIs

Grant all three to anyone running an agentic loop. A developer holding only `system:mcp-consumer` can browse the catalog and call tools by hand, and will then hit a refusal the moment their agent tries to reach a model — which reads as a broken platform rather than a missing role.

> **Note.** These roles are grant *targets*, not capability bundles. Holding `system:llm-consumer` does not by itself reach any provider; access to each one is granted explicitly. The role is what makes that grant possible.

6.2 Send them to the Developer Portal

The portal is a separate surface from the admin UI, with its own guide: the *Magertron Developer Portal Guide*. Point them there rather than at this document. What they will find:

- **The catalog** — every server and tool their roles permit, and nothing else. The list is filtered by their grants, so it is already an accurate statement of their reach.
- **Tool schemas** — arguments and types, as discovered from the server itself rather than transcribed by hand.
- **A live session** — mint a token, make a call, and see both the result and the audit record it produced.

That last point is worth emphasising to them: **the first successful tool call is the moment the platform proves itself.** A server that shows as `Running` has not yet demonstrated anything.

6.3 What the developer does not need

Worth saying plainly, because it saves a conversation:

- **They do not need the admin UI.** If they are asking for access to it, either they need a governance change made on their behalf, or they have been given the wrong starting point.
- **They do not need to know about namespaces, licensing, or governance policies.** Those shape what they can reach; they do not need to reason about them to work.
- **They do not need credentials for the upstream vendors.** That is the point of the platform — the gateway injects them. A developer who is being handed an API key is bypassing the thing you installed.

6.4 What this guide has and has not proven

You now have a platform that installs, a server that runs, and a developer who can call it. That is the whole first-run path.

It has not been under load, and it has not been governed yet. Both are deliberate: a first install with policies enforcing and budgets throttling would fail for reasons that have nothing to do with whether the install worked. When you are ready for them, they are the next two things to configure.

- **Governance policies** — the platform ships two templates, `enterprise-standard` and `dev-relaxed`, both disabled. They are starting points to adopt and edit, not defaults to inherit. Enable one deliberately once you have decided your own conventions; `enterprise-standard` requires a `team` label on every server, which is a reasonable rule and not one you want arriving unannounced.
- **Cost contracts and budgets** — metering runs from the first call, so the usage data is already accumulating whether or not you have priced it.

6.5 Uninstalling

To remove Magertron and everything it deployed:

```
helm uninstall mcp -n mcp-system
kubectl delete namespace mcp-system mcp-prod
```

The Helm release is named `mcp`. Deleting both namespaces removes the control plane and the MCP servers it deployed. Wait for them to disappear before reinstalling — an install started while they are still terminating fails:

```
kubectl get namespace | grep mcp
```